Finite State Machines

Original slides by: David Wilkinson
Modified with bits from: Bruce Childers, and Jarrett Billingsley

#11

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Fall 2020

Finite State Machines

What's an FSM?

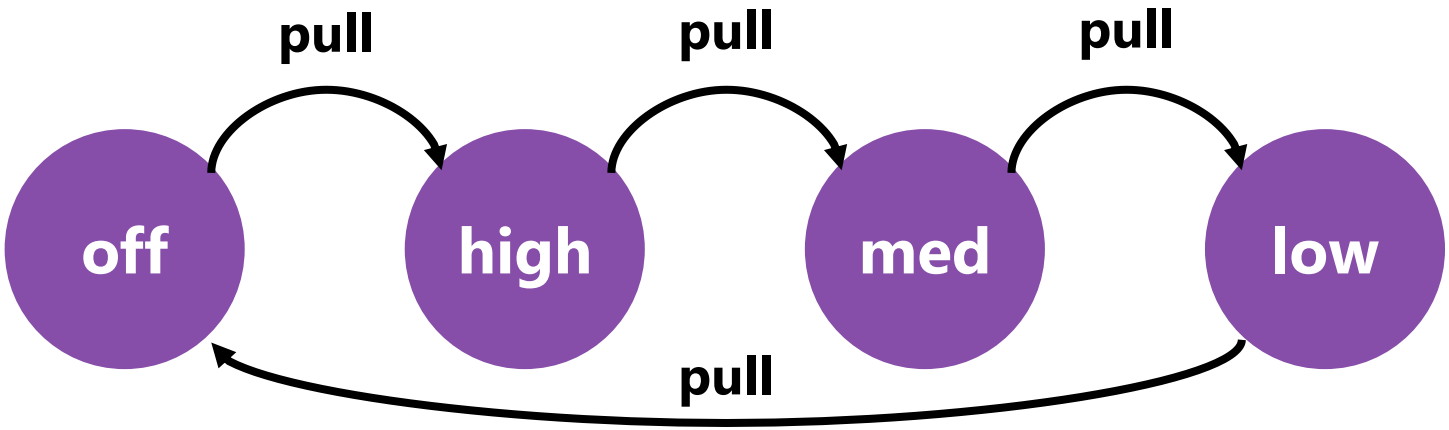
- A **Finite State Machine** is a machine with a finite number of states.
  - :')
- A Finite State Machine is a way of thinking about a process where:
  - there is a series of **inputs**
  - we need to produce a series of **outputs**
  - the **state** stores some information
  - the inputs can **change the state and the outputs**

- FSMs come up all the time in programming and hardware design
- They're great for controlling simple multiple-step procedures

A very simple example

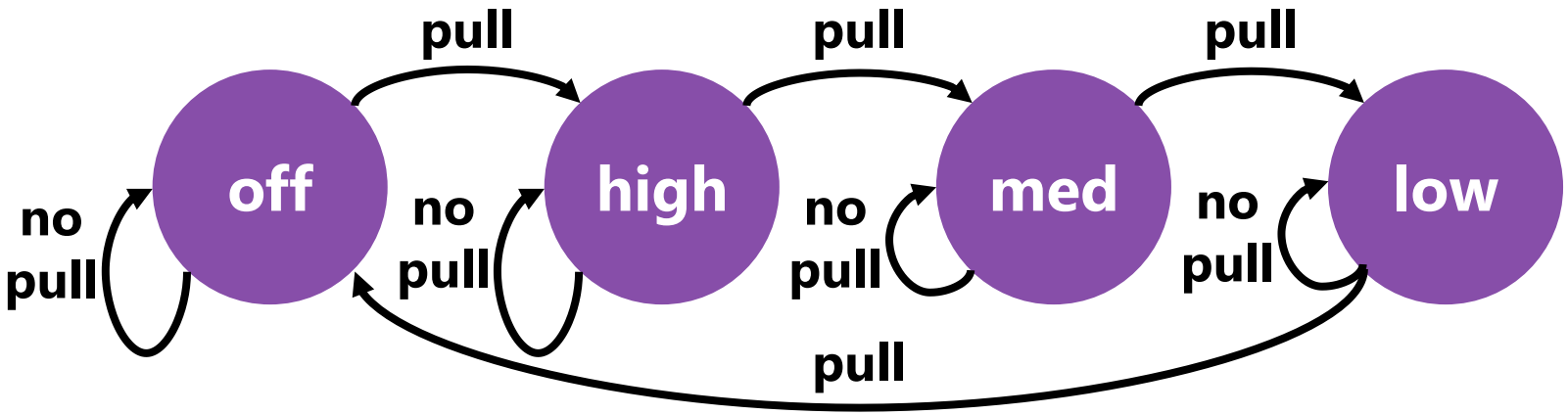
- what states can a ceiling fan be in?
  - high, medium, low, and off
- what are the input and output?
  - the chain and the motor
- when you pull the chain, it **changes state**





Missing some arrows

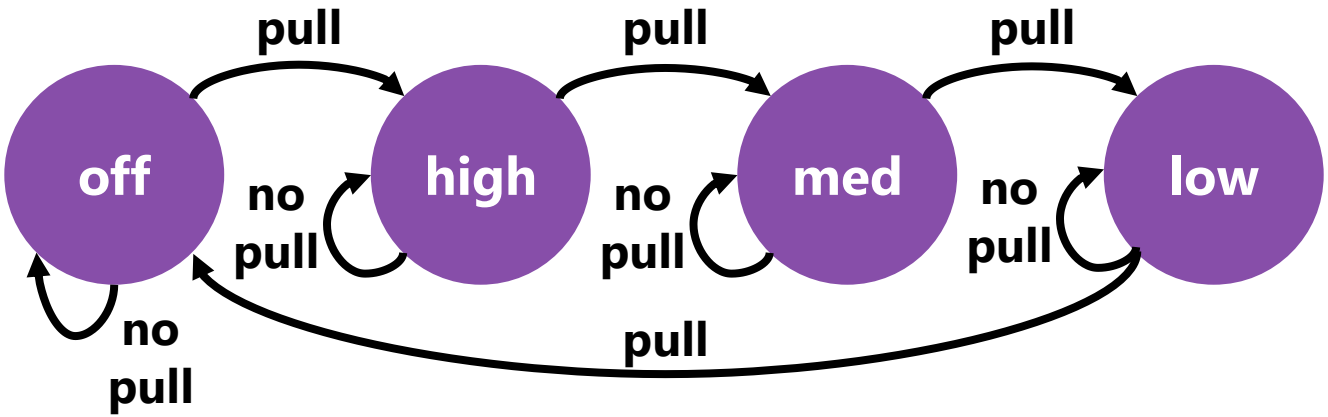
- at any point in time, **which of 2 choices can the input (chain) be?**
  - pulled or **not-pulled**
- when you *don't* pull the chain, what happens?



this is the **state transition diagram**

The state transition table - Table-a-fyin' it

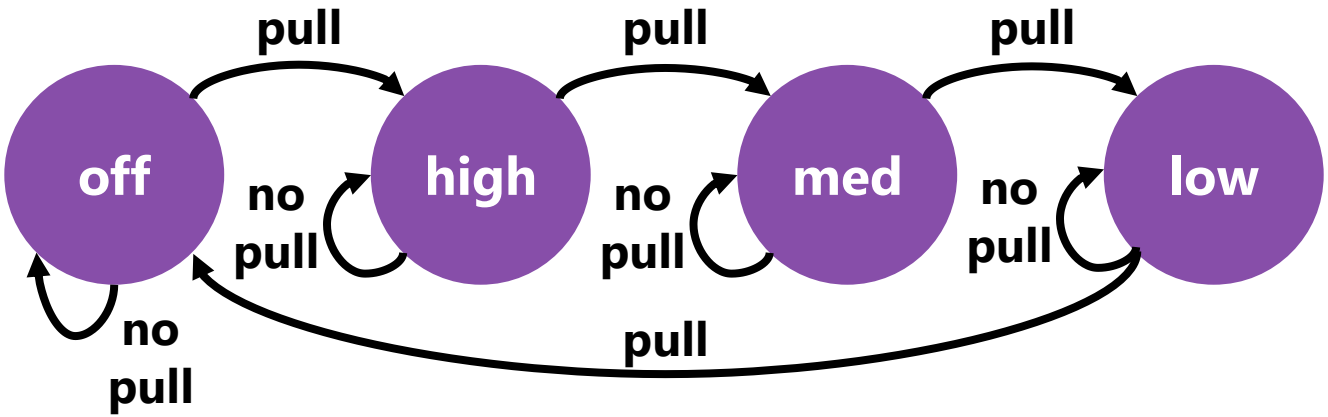
- we can represent this diagram with a truth table



S	In	S_{next}
off	no pull	off
off	pull	high
high	no pull	high
high	pull	med
med	no pull	med
med	pull	low
low	no pull	low
low	pull	off

The state transition table - Table-a-fyin' it

- we can represent this diagram with a truth table
- we have 4 states. **how many bits** are needed to represent 4 values?

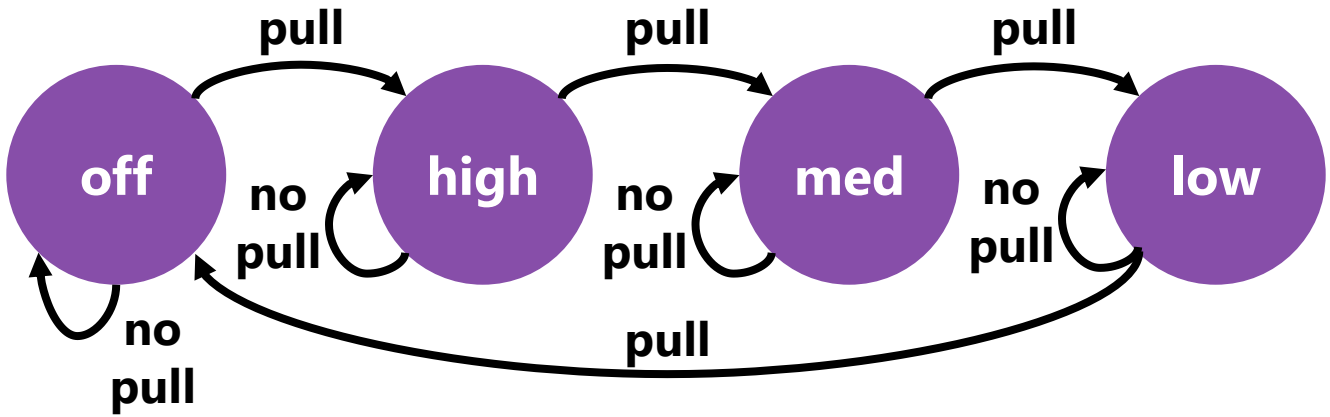


for the state, let's say:
off means **00**
high means **01**
med means **10**
low means **11**

S	In	S_{next}
off	no pull	off
off	pull	high
high	no pull	high
high	pull	med
med	no pull	med
med	pull	low
low	no pull	low
low	pull	off

The state transition table - Table-a-fyin' it

- we can represent this diagram with a truth table
- we have 4 states. **how many bits** are needed to represent 4 values?

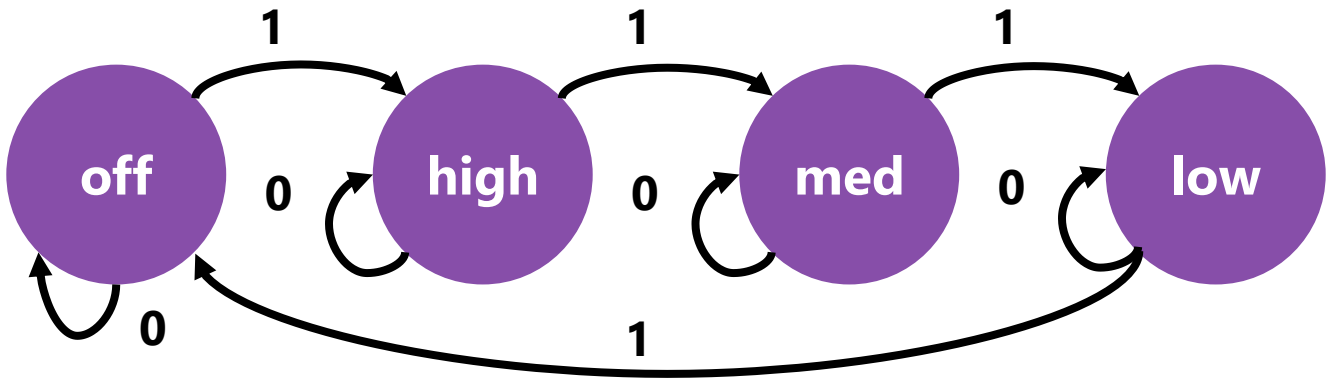


for the input, let's say:
0 means **not pulling**
1 means **pulling**

S	In	S_{next}
00	no pull	00
00	pull	01
01	no pull	01
01	pull	10
10	no pull	10
10	pull	11
11	no pull	11
11	pull	00

The state transition table - Table-a-fyin' it

- we can represent this diagram with a truth table
- we have 4 states. **how many bits** are needed to represent 4 values?



for the input, let's say:
0 means **not pulling**
1 means **pulling**

S	In	S_{next}
00	0	00
00	1	01
01	0	01
01	1	10
10	0	10
10	1	11
11	0	11
11	1	00

It's a ~~UNIX system~~ truth table, I know this!

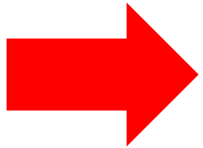


Image from Jurassic Park (1993), distributed by Universal Pictures

Truth tables with multiple output bits

- there's no tricky stuff here – each output bit is **its own expression**

S	In	S_{next}
00	0	00
00	1	01
01	0	01
01	1	10
10	0	10
10	1	11
11	0	11
11	1	00



S_1	S_0	In	S_{next1}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

S_1	S_0	In	S_{next0}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

What about the outputs

- our fan controller has to control **the motor** after all
- we can make a table showing the output(s) for **each state**

Assume this is how you control the motor

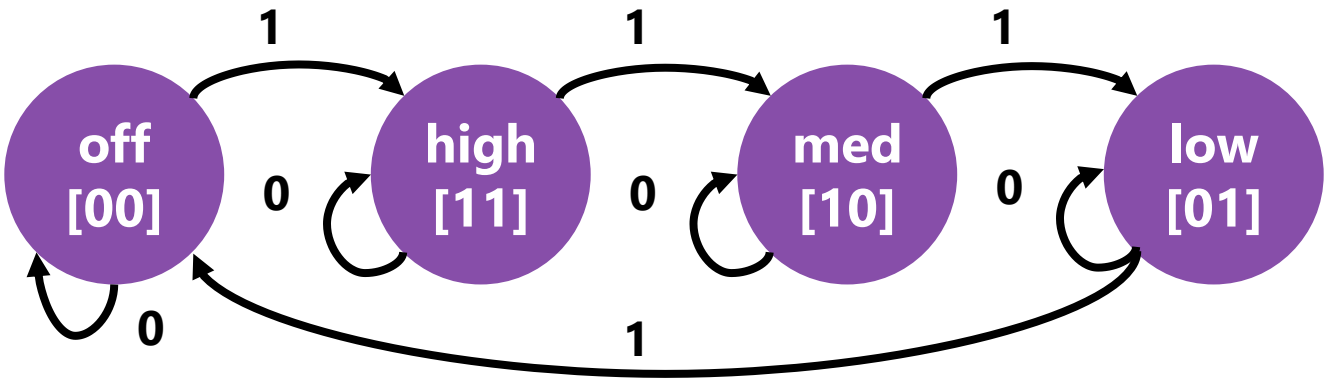
note that the input isn't involved at all

the outputs only have to depend on the **current state!**

State	Motor power	Output
off	0%	00
high	100%	11
med	50%	10
low	25%	01

Representing the output

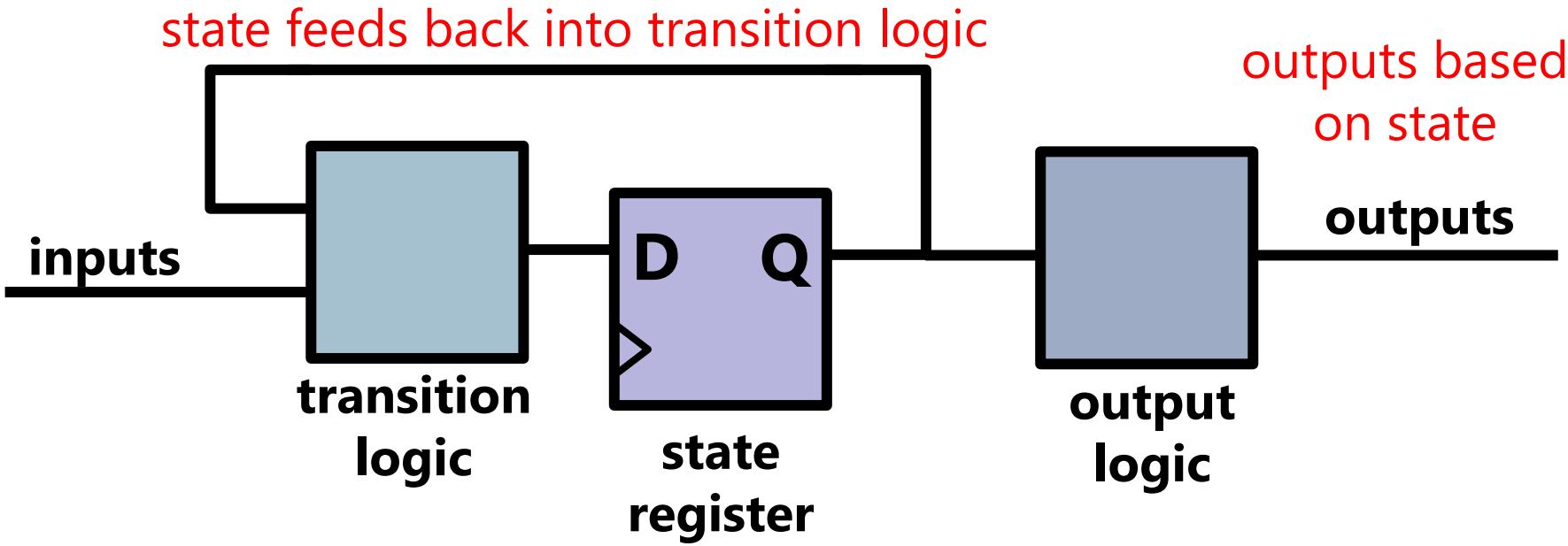
- We can represent the output in the state machine!



S	Output
00	00
01	11
10	10
11	01

Making a circuit out of it

- This is a **sequential circuit –** the state changes over time
- But the state transition and output tables are just **combinational**
- Here's the general organization of any Moore FSM circuit:



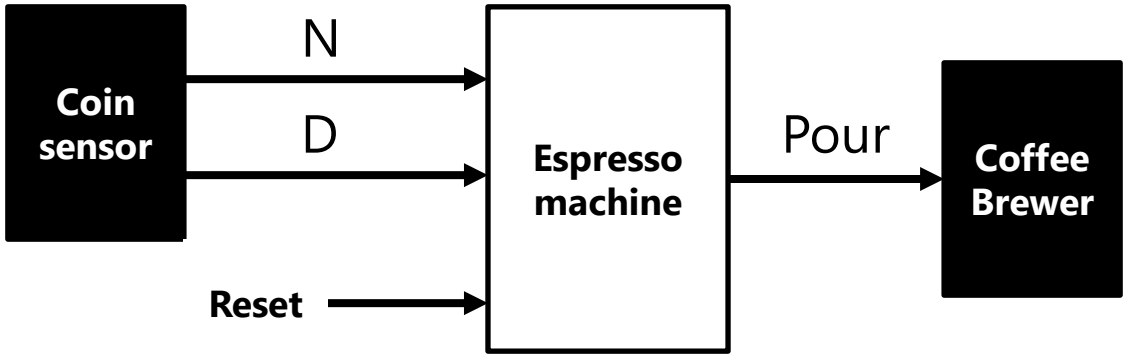
How to create a state machine?

- Understand the problem
- Represent all possible states and transitions
- Encode the states
- Implement the machine

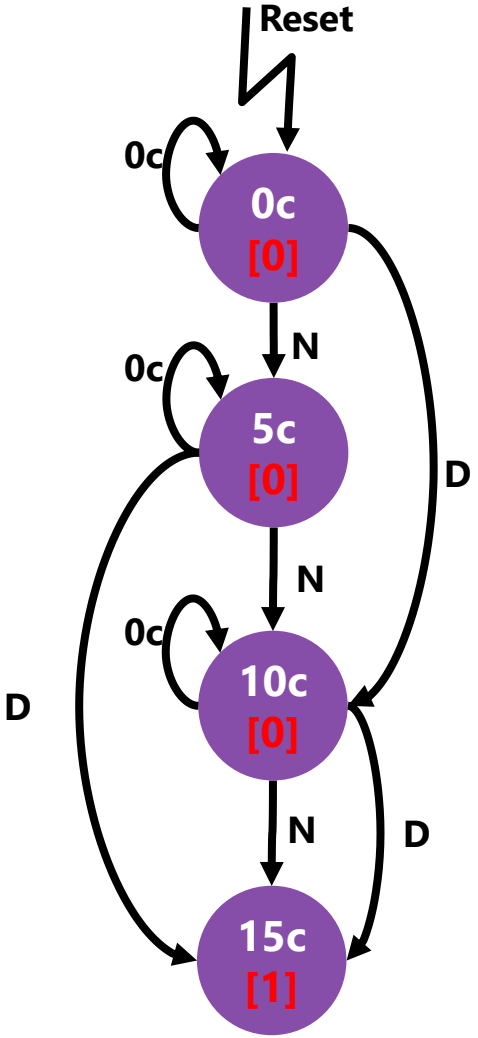
Understanding the problem

A problem:

- You have an espresso machine
  - Each espresso costs 15c **<3**
  - The machine takes:
    - **5c** - (N)ickels
    - **10c** - (D)imes
  - No change is given back!



Represent all possible states and transitions



S	D	N	S_{next}	Output
0c	0	0	0c	0
0c	0	1	5c	0
0c	1	0	10c	0
0c	1	1	X	X
5c	0	0	5c	0
5c	0	1	10c	0
5c	1	0	15c	0
5c	1	1	X	X
10c	0	0	10c	0
10c	0	1	15c	0
10c	1	0	15c	0
10c	1	1	X	X
15c	X	X	15c	1

Encode the states

S	Encoded
0c	00
5c	01
10c	10
15c	11



S	D	N	S_{next}	Output
0c	0	0	0c	0
0c	0	1	5c	0
0c	1	0	10c	0
0c	1	1	X	0
5c	0	0	5c	0
5c	0	1	10c	0
5c	1	0	15c	0
5c	1	1	X	0
10c	0	0	10c	0
10c	0	1	15c	0
10c	1	0	15c	0
10c	1	1	X	0
15c	X	X	15c	1

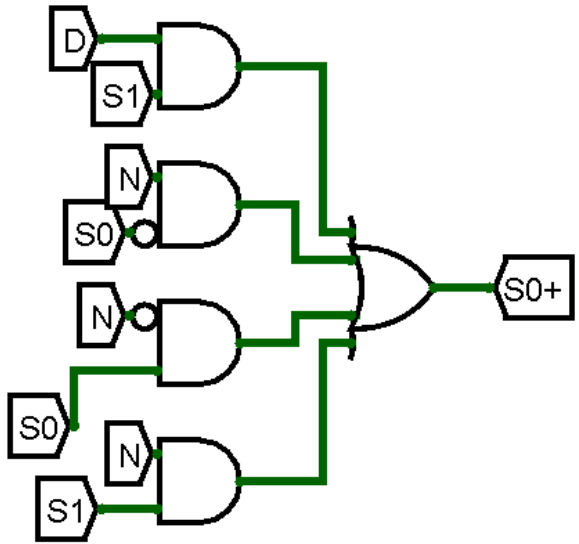
Our good old friend K-map - S_{next0}

DN \ S_1S_0	00	01	11	10
00	0	1	1	0
01	1	0	1	1
11	X	X	X	X
10	0	1	1	1

Red: DS_1 **Green: $N\overline{S_0}$**

Blue: $\overline{N}S_0$ **Purple: NS_1**

$$S_{next0} = DS_1 + N\overline{S_0} + \overline{N}S_0 + NS_1$$



Our good old friend K-map - S_{next1}

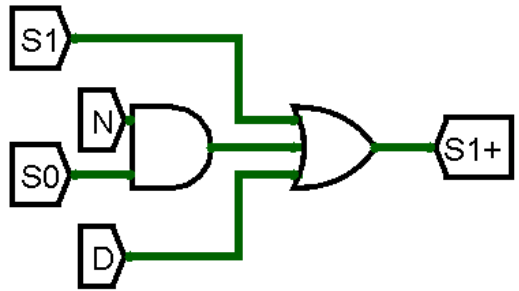
DN \ S_1S_0	00	01	11	10
00	0	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	1	1

Red: S_1

Green: NS_0

Blue: D

$$S_{next1} = S_1 + NS_0 + D$$

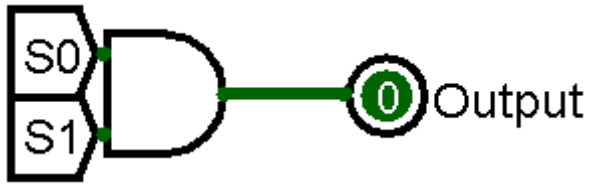


Our good old friend K-map - Output

S_1 \ S_0	0	1
0	0	0
1	0	**1**

Red: S_0S_1

Output= S_0S_1



Get everything together

